

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS700 Research Report

A Per-User Behavioural Baseline Framework for Insider Threat Detection in Resource-Constrained Environments

Supervisor(s):
Prof. Jan Eloff

Selepe Sello (uXXXXXXXX)

October 2026

Abstract

Insider threats are slow to detect and costly: in 2024, breaches by malicious insiders took an average of 287 days to identify and contain. Lightweight detectors that score every employee against one organisation-wide baseline fail in two directions at once. Legitimately busy users such as IT administrators fill the alert queue, while an insider whose behaviour changes slowly, within population norms, is never surfaced. This report designs, implements and evaluates a lightweight per-user behavioural baseline framework. It profiles each user from their own history, falls back to a role-group profile when history is short, and scores each day's deviation vector with a single unsupervised anomaly model.

The framework was evaluated on the full CERT Insider Threat Test Dataset r6.2 (4 000 users, 135 million log events) at an alert budget of 0.1% of user-days. It found 21 of 38 low-and-slow malicious days and both test-window insiders, against 8 days and one insider for the SMOTE-balanced Random Forest that represents current practice. It spent 3.5% of its alerts on IT administrators instead of 22.6%, and trained in 8 seconds instead of 447. The same anomaly algorithm with a population reference found 3 days, which isolates the gain to the per-user reference frame. Across ten seeds the framework found 20 or 21 of the 38 days every time, and its advantages in recall and administrator alert share over every population detector held in all ten (Wilcoxon signed-rank, Holm-adjusted $p = 0.035$). Follow-up studies show that a personal baseline is stable after about 30 days of history and useful after about 60. Its explanations name a feature the insider actually used as the top reason on 95% of malicious days, against 50% for the Random Forest's SHAP explanations read at their best. The test window holds only two insiders and the data is synthetic, and both limits are discussed alongside the results.

Keywords:

insider threat detection, behavioural anomaly detection, per-user baseline, user and entity behaviour analytics, role-aware normalisation, alert fatigue, CERT r6.2, one-class SVM, explainability, resource-constrained deployment.

Contents

1 Introduction **5**

2 Problem Statement **7**

3 Background **9**
3.1 Insider Threats and the CERT r6.2 Dataset 9
3.2 Behavioural Baselines and Normalisation 9
3.3 Unsupervised Anomaly Detection 10
3.4 Supervised Population Baselines and Class Imbalance 11
3.5 Evaluation Under Extreme Class Imbalance 11
3.6 Explainability 12
3.7 Statistical Comparison of Detectors 12

4 Related Work **13**

5 Methodology **14**
5.1 Data Preparation 15
5.2 The Per-User Behavioural Baseline Framework 17
5.3 Comparison Set 18
5.4 Evaluation Protocol 19
5.5 Follow-up Studies 20
5.6 Implementation and Parameters 21
5.7 Validation of the Implementation 23

6 Results and Discussion **24**
6.1 Headline Comparison 24
6.2 SQ1: Low-and-Slow Detection 26
6.3 SQ2: False Positives on High-Privilege Users 28
6.4 Ablation: Reference Frame, Statistic, Algorithm and Features 30
6.5 Robustness Across Seeds 31
6.6 SQ3: How Much History a Personal Baseline Needs 32
6.7 SQ4: Computational Cost 34
6.8 SQ5: Context for the Analyst 35
6.9 Unexpected and Negative Results 38
6.10 Threats to Validity 38

7 Conclusion **39**

A Reproducing the Results **45**

1 Introduction

Insider threats are among the slowest security incidents to detect and the most expensive to suffer. In IBM's 2024 study of 604 breached organisations, attacks by malicious insiders took an average of 287 days to identify and contain, against 258 days for breaches overall, and were the costliest initial attack vector at an average of USD 4.99 million [1]. The difficulty is structural. An insider already holds legitimate credentials and legitimate access, so perimeter controls do not apply, and the detection problem reduces to distinguishing malicious use of that access from routine use of it [2, 3].

The lightweight detectors that most organisations can afford, and much of the research on the CERT data [4, 5], solve that problem by learning one model of "normal" from the whole workforce and flagging whoever departs from it. A single shared baseline is cheap and simple, but it fails in two directions at once. Employees whose work is naturally intense, such as IT administrators who touch many machines, connect devices and work odd hours, are persistently unusual relative to the population, so they fill the alert queue while doing their jobs. Analysts who see the same false alarms every day learn to discount them [6]. Conversely, an insider who changes their behaviour gradually, and keeps every individual day within what the population considers ordinary, is never surfaced at all, even when the change is dramatic relative to their own history. This is the "low-and-slow" pattern of data theft documented in the CERT insider threat casebook [2].

Commercial User and Entity Behaviour Analytics (UEBA) platforms address this by profiling each user and their peer group individually [7]. Academic work has likewise moved towards user-centred and per-user modelling of the CERT data [8, 9]. Both routes, however, assume infrastructure that many organisations do not have: a SIEM feeding a licensed analytics platform, or a deep sequence model trained on streaming logs. There is no published, lightweight implementation that builds per-user baselines from static log exports on ordinary hardware and is evaluated against the population-level practice it would replace.

This report designs, implements and evaluates such a framework. Each user's normal behaviour is profiled from their own history and from their role group, the day's activity is expressed as a vector of deviations from that profile, and a single unsupervised anomaly model scores those deviation vectors. Because the anomaly model sees "how unusual is this for this person" rather than raw activity, one cheaply trained model behaves as a per-user detector. The framework is evaluated on the full CERT Insider Threat Test Dataset r6.2 [10, 11] (4 000 users, 135 million log events) against the Random Forest that prior work identifies as the strongest population-level classifier

for this data [4], and against controls that isolate the per-user reference frame as the cause of any difference.

Contributions. This report makes the following contributions:

1. A lightweight per-user behavioural baseline framework, comprising a Profile Builder, a Role-Aware Normaliser and an Anomaly Scorer. It runs on a laptop and trains on the full 4 000-user dataset in under ten seconds.

2. A streaming data pipeline that reduces the 93 GB of raw r6.2 logs to an 8.7 MB, self-contained, user-day dataset in 3.3 minutes on 32 GB of RAM, with ground truth derived from the official answer key.

3. A controlled comparison of 45 detector configurations on one chronological split, at a fixed alert budget rather than a shared threshold. The comparison includes a same-algorithm population control, a trivial volume floor, and the four population classifiers compared by Noever [4].

4. Three follow-up studies that close gaps the headline comparison leaves open. The first re-runs the key detectors across ten seeds, with paired Wilcoxon signed-rank tests and Holm-Bonferroni correction. The second measures how many days of history a personal baseline needs (subquestion 3). The third scores each system's explanations against the features the official answer key says the insider actually used (subquestion 5).

5. A report of the negative and unexpected results alongside the positive ones: a volume floor that beats most published-style baselines, two supervised classifiers that fail outright, and design choices that recreate the very failure the research set out to remove.

Scope and findings. The framework outperforms the population Random Forest on every comparison the benchmark supports. It finds 2.6 times as many low-and-slow malicious days, surfaces both test-window insiders, and spends a sixth as much of the alert queue on administrators. It also trains 57 times faster. A same-algorithm control attributes these gains to the per-user reference frame rather than to the anomaly model. These differences held in every one of ten seeds. The follow-up studies add two practical findings. A personal baseline needs about 60 days of history before it beats the role baseline, so the proposal's 14-day cold-start threshold is too low. And the

framework's explanations name what the insider actually did on 95% of malicious days, against at most 50% for the Random Forest's SHAP values; read by absolute value, the SHAP explanations do worse than random. The evidence has clear limits. CERT r6.2 is synthetic, its chronological test window contains only two insiders (both low-and-slow), and the anomaly algorithm was selected on data that contains them. Section 6.10 discusses each limit and what it does and does not undermine.

2 Problem Statement

Organisations that cannot afford enterprise security platforms rely on detectors that evaluate every user against one baseline derived from the whole workforce. Two concrete failures follow. First, a legitimately high-activity user, such as an IT administrator who logs on to many machines and works after hours, is flagged because their activity is unusual for the population, although it is normal for them and for their role. Repeated false alerts of this kind teach analysts to ignore the system [6]. Second, an insider who spreads data theft across many ordinary-looking days is not flagged, because no single day exceeds a population threshold, even though their behaviour has changed sharply relative to their own history [2].

Both failures have the same cause: the detector has no model of what is normal *for this individual in this role*. The knowledge gap addressed is therefore the absence of a published, lightweight framework that constructs individual behavioural baselines from standard organisational log exports, normalises activity relative to a user's role group, and produces anomaly scores that separate genuine individual deviations from role-consistent behaviour, without enterprise infrastructure.

The central hypothesis is that scoring each user-day as a deviation from the user's own baseline, with role-group fallback, finds more low-and-slow insider activity and spends fewer alerts on legitimate high-privilege users than a population-level baseline, at a computational cost that is feasible on consumer hardware. The research question breaks down into the five sub-questions of the proposal:

SQ1 To what extent does a per-user behavioural baseline reduce missed detections of low-and-slow insider behaviour compared to a population-level baseline?

SQ2 To what extent does role-aware normalisation reduce the false positive rate for high-privilege legitimate users compared to population-level evaluation?

SQ3 What is the minimum amount of historical log data needed to construct a stable and reliable individual baseline?

SQ4 How does the computational cost of per-user baseline construction compare to population-level model training, and is it feasible in a resource-constrained deployment?

SQ5 What additional context does a per-user deviation score give a security analyst relative to a population-level score, and does it reduce investigation time?

Scope. The research uses behavioural features derived from organisational log files only. It excludes packet-level network traffic, the content of email and web pages, psychometric data, and real-time streaming deployment. The evaluation uses a single public benchmark, CERT r6.2. It assumes that logs are available as structured exports and that role assignments are recorded. Role misclassification and deliberate manipulation of role labels are out of scope.

Deviations from the proposal. Four changes were forced by the data or by what the experiments revealed:

- **Dataset size and log types.** The proposal described r6.2 as 1 000 users over 18 months with print logs. The release actually contains 4 000 users over 516 days, and has no print log. It does have logon, removable-device, file, web (HTTP) and email logs, all of which are used, together with monthly LDAP snapshots for roles. Email and web *content* is not used, only metadata such as counts, recipients, sizes and upload actions.
- **Operating point.** The proposal's metrics (recall, high-privilege false positive rate, F1, ROC-AUC) are all reported, but at a fixed *alert budget* rather than a fixed threshold, and with average precision as the primary ranking metric. Section 5.4 explains why the benchmark forces this.
- **Temporal features.** The proposal's risk register anticipated adding rolling-window features if the base framework underperformed. They were implemented and evaluated, and they help one anomaly algorithm and hurt another (Section 6.4).
- **Investigation time (SQ5).** The proposal asked whether per-user scores reduce investigation time. A user study was not feasible within

the project, so SQ5 is answered with two measurable proxies: the queue depth an analyst must work through before reaching each insider, and the fidelity of each system's explanations against ground truth.

3 Background

3.1 Insider Threats and the CERT r6.2 Dataset

An insider threat is a current or former employee, contractor or partner who misuses authorised access to harm the organisation [2]. Surveys of the field group detection approaches by the data they model and distinguish misuse that is abrupt from misuse that unfolds slowly [3, 12]. Real labelled insider data is almost never released, so research relies on synthetic corpora. The most widely used is the CERT Insider Threat Test Dataset [10], generated by the CERT Division and ExactData from behavioural models of a fictitious organisation, with malicious scenarios injected by red-team narratives [11].

Release r6.2, used here, covers 4 000 employees from 2 January 2010 to 1 June 2011. It contains five event logs (logon/logoff, removable-device connect/disconnect, file operations, web browsing and email) and eighteen monthly LDAP snapshots that record each employee's role, department and supervisor. There are 46 distinct roles. An answer key lists every malicious log line. In r6.2 one insider enacts each of five scenarios, summarised in Table 1 from the answer key's own events.

The benchmark is harder than its size suggests. Only 44 of 1 394 010 user-days are malicious, about one in 31 700. As Section 5.4 shows, the insiders are also not spread evenly across the calendar.

3.2 Behavioural Baselines and Normalisation

A behavioural baseline is a description of normal activity against which a new observation is compared. For a feature x_f observed on one user-day, the standard score against a reference distribution with mean μ_f and standard deviation σ_f is

$$z_f = \frac{x_f - \mu_f}{\sigma_f}.$$

The choice of reference distribution is the subject of this research. A *population* baseline estimates μ_f and σ_f over every employee. A *role* baseline estimates them over the employee's role group. A *personal* baseline estimates them over the employee's own history.

Table 1: The five r6.2 insiders, as recorded in the official answer key. "Days" counts distinct malicious user-days; "span" runs from the first malicious event to the last.

Insider	Role	Malicious activity	Days	Span	Split
ACM2278	Salesman	After-hours logons, removable drive, uploads to wikileaks.org	3	6 d	train
CMP2946	Salesman	Job-site browsing, job-search email to competitors, thumb-drive use	29	56 d	test
PLJ1771	ITAdmin	Keylogger web visits, removable drive, logons, mass internal email	2	1 d	train
CDE1846	Elec. Engineer	Logons to other users' PCs, file opens, documents emailed to a personal address	9	63 d	test
MBG3183	Elec. Engineer	Four uploads in one minute	1	0 d	train

Behavioural counts are zero-inflated and right-skewed: most users never write to removable media, and when they do it is in bursts. A single large day inflates σ_f and masks the next one. The robust alternative replaces the mean with the median and the standard deviation with the interquartile range scaled to be consistent with σ under normality [13]:

$$z_f^{\text{robust}} = \frac{x_f - \text{median}_f}{\text{IQR}_f \,/1.349}.$$

Both forms are evaluated rather than assumed.

3.3 Unsupervised Anomaly Detection

Labelled insider activity is scarce, so anomaly detectors that learn only from (mostly) normal data are a natural fit [14]. Three are compared, as the proposal specifies.

Isolation Forest [15] isolates points by recursive random axis-parallel splits. Anomalies are isolated in fewer splits, so with $h(x)$ the path length of x in a tree and $c(n)$ the average path length of an unsuccessful binary search over n points, the anomaly score is

$$s(x, n) = 2^{-\mathbb{E}[h(x)]/c(n)},$$

which approaches 1 for anomalies. It is near-linear in the training set and tolerant of redundant features, because each split samples one feature.

Local Outlier Factor (LOF) [16] compares the local density around a point with the density around its k nearest neighbours. Points in regions much sparser than their neighbours' receive high scores. Its cost is dominated by nearest-neighbour search.

One-Class SVM [17] maps the data into a feature space with a kernel $k(\cdot, \cdot)$ and finds the hyperplane that separates the data from the origin with maximum margin. A parameter $\nu \in (0, 1]$ bounds the fraction of training points treated as outliers. The decision function is

$$f(x) = \operatorname{sgn}\Big(\sum_i \alpha_i k(x_i, x) - \rho\Big),$$

and the signed distance $\sum_i \alpha_i k(x_i, x) - \rho$ serves as a continuous score. With an RBF kernel, $k(x, x') = \exp(-\gamma \|x - x'\|^2)$, the model is sensitive to the geometry of the whole feature space. This matters when features are added (Section 6.4). Training is quadratic in the number of rows.

3.4 Supervised Population Baselines and Class Imbalance

Supervised classifiers learn the malicious class directly from labelled examples. On the CERT data, Noever compared 88 algorithms from 16 families and found Random Forests the strongest [4], which is why the proposal designates a Random Forest as the population baseline. With malicious activity below 0.02% of events, training sets are usually rebalanced. The Synthetic Minority Over-sampling Technique (SMOTE) creates synthetic minority examples by interpolating between a positive example and one of its k nearest positive neighbours [18]. It is applied to the training split only, after the split, so no synthetic row can leak information from the test window.

3.5 Evaluation Under Extreme Class Imbalance

ROC-AUC and average precision. ROC-AUC measures the probability that a random positive is ranked above a random negative. When negatives outnumber positives 31 000 to one, a detector can rank the benign mass well, and so score a high ROC-AUC, while placing no positive anywhere near the

top of the queue [19, 20]. Average precision (AP) summarises the precision-recall curve instead:

$$\mathrm{AP} = \sum_n (R_n - R_{n-1})\, P_n,$$

where P_n and R_n are precision and recall at the n-th threshold. Its chance level equals the positive rate, here about 10^{-4}, and it is dominated by the top of the ranking, which is the only part an analyst ever reads.

Alert budgets. A Random Forest emits a probability, an Isolation Forest a path-length score and a One-Class SVM a margin distance. Thresholding all three at 0.5 compares a probability with two arbitrary constants. The comparison is therefore made at an *alert budget*: each detector may flag the top b fraction of test user-days. This is also how detection is deployed, since an analyst team has a fixed capacity to investigate, not a fixed decision threshold. At a budget, recall, precision and F1 are well defined for every detector. *Lift*, the precision divided by the positive rate, says how many times better than random the alerts are.

3.6 Explainability

A flagged user-day with no reason attached must be investigated from scratch. SHAP attributes a model's output $f(x)$ to its input features using Shapley values from cooperative game theory [21]. The contribution of feature i is

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \Big[f_{S\cup\{i\}}(x_{S\cup\{i\}}) - f_S(x_S) \Big],$$

and TreeExplainer computes these values exactly and efficiently for tree ensembles [22]. The per-user framework has a more direct explanation available: the deviation vector itself. Its largest components name the features on which today departs most from this user's normal.

3.7 Statistical Comparison of Detectors

Several components of every detector are random, including subsampling, forest construction, SMOTE and tie-breaking. A difference observed under one seed may therefore be an artefact of that draw. Re-running each detector under n seeds yields paired observations: under seed s, detectors A and B give metric values a_s and b_s. The Wilcoxon signed-rank test [23] ranks the non-zero differences $|a_s - b_s|$ and tests whether the positive and negative

ranks are balanced, without assuming normality. It is the test recommended for comparing classifiers over paired runs [24]. With $n = 10$ seeds and every difference in the same direction, the exact two-sided p-value is $2/2^{10} \approx 0.002$, which is the smallest the test can produce at this n. When m hypotheses are tested together, the Holm-Bonferroni procedure [25] controls the family-wise error rate. It compares the i-th smallest p-value with $\alpha/(m-i+1)$ and stops at the first failure.

4 Related Work

Machine learning on the CERT data. The CERT releases are the standard benchmark for insider threat detection [3, 12]. Noever scored 88 classifiers from 16 families on CERT-derived risk features and found Random Forests the most accurate, reporting over 98% accuracy and noting that the class balance (below 0.02% malicious) required both under- and over-sampling [4]. Le and Zincir-Heywood built a detection workflow combining Hidden Markov Models, Self-Organising Maps and decision trees [5]. Le et al. then showed that the granularity at which activity is aggregated (per user-day, per user-week or per session) materially changes what machine-learning detectors can find [9]. Yuan et al. replaced hand-crafted features with an LSTM that learns temporal features from user action sequences, followed by a CNN classifier [26]. All of these model the population: one decision boundary, learned from every user, applies to every user. The present work keeps the user-day granularity of Le et al. and the Random Forest of Noever as its comparison point, and changes only the reference frame.

Per-user and user-centred modelling. Tuor et al. evaluated online unsupervised deep and recurrent networks on CERT r6.2, decomposed each anomaly score into the contributions of individual behaviour features, and reported that their models outperformed Isolation Forest, PCA and SVM baselines [8]. Their evaluation is per-event and per-user rather than at a population threshold. Commercial UEBA platforms take the same idea into production: Microsoft Sentinel, for example, builds behavioural profiles for each user and compares activity with both the user's own history and their peers' [7]. Both routes require infrastructure (GPU training on streaming data, or a licensed SIEM and analytics platform) that a resource-constrained organisation does not have. The framework here pursues the same per-user idea with descriptive statistics and one classical anomaly model, on static exports.

Alert fatigue and the analyst. In interviews with security operations centre analysts, Alahmadi et al. found false positives to be the dominant complaint about security alarms, and found that analysts come to distrust tools that raise them [6]. This motivates measuring *who* an alert budget is spent on, rather than only an aggregate false positive rate.

Explainability. SHAP [21] and its exact tree variant [22] are the default way to explain a tree ensemble's individual predictions, and are what an analyst using a Random Forest detector would be shown. Explanations in insider threat work are usually evaluated qualitatively. This report instead scores them against the answer key.

Forensic readiness. Shoderu et al. argue that insider-threat mitigation in SaaS environments remains largely reactive, and propose embedding digital forensic readiness, aligned with ISO/IEC 27043, so that anomalies are detected early and evidence is captured in a legally defensible way [27]. A per-user baseline that explains each alert in terms of the user's own history is one practical source of such early, attributable signals.

Gap. The weakness of population baselines for power users and slow insiders is recognised implicitly throughout this literature. The per-user approach that addresses it exists either as deep sequence models or as enterprise platforms. No published study implements a lightweight per-user baseline that runs on static log exports on consumer hardware, compares it with the population-level practice it would replace under a controlled design, and reports both its detection and its explanations against ground truth. This report fills that gap.

5 Methodology

The research uses a quantitative, controlled experimental design. A per-user framework and a set of population-level detectors are trained on identical data, scored on an identical held-out window, and compared on predefined metrics at a shared operating point. Engineering the pipeline and the framework followed an iterative prototyping cycle. Each iteration was validated by regression tests that pin specific ways an imbalanced-data comparison can report a misleading number (Section 5.7). Figure 1 summarises the stages.

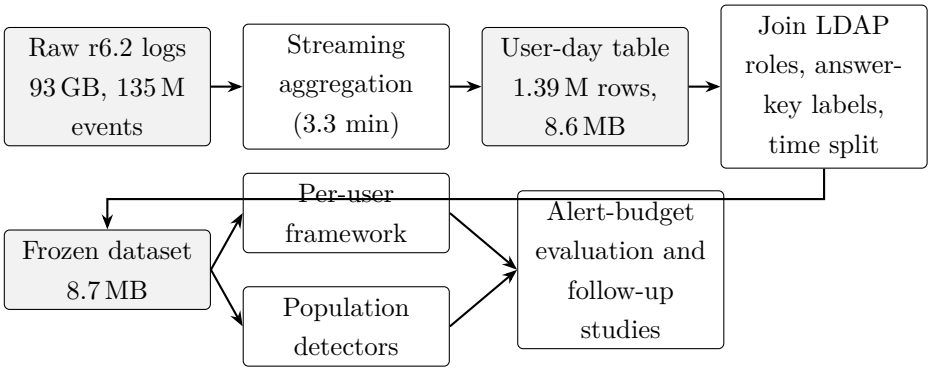


Figure 1: The pipeline. The raw logs are read once, reduced to one row per user and day, and frozen with roles, labels and the train/test split into a single 8.7 MB file. Every detector is trained and evaluated from that file alone.

5.1 Data Preparation

Streaming aggregation. The extracted r6.2 logs total 93.2 GB, of which the web log alone is 84 GB, and cannot be loaded into 32 GB of memory. Every feature the framework uses is a per-user, per-day count, sum or distinct count, and these can be computed in one sequential pass with memory proportional to the number of user-days rather than the number of events. The aggregator splits each file into byte ranges snapped to record boundaries and parses the shards in parallel. It extracts the calendar day and hour by slicing the fixed-width timestamps instead of parsing them, and merges per-shard counters. On 14 cores this reduces the 135 million events to 1 394 010 user-days in 3.3 minutes. Regression tests require the sharded result to equal a sequential pass exactly.

Features. Each user-day carries thirteen behavioural features (Table 2). Four of them were constant against the real data in an early version: the removable-media flag, external-email detection, send-versus-view filtering, and distinct domains versus distinct URLs. They were corrected and pinned by tests before any experiment was run. An email is external if any recipient lies outside the organisation's domain, which is learned from the LDAP directory. This matters because about a quarter of r6.2's sent mail originates from personal webmail accounts on company machines.

Table 2: The thirteen per-user-day behavioural features.

Log	Feature	Definition
logon	n_logon, n_logoff	Logon and logoff events
	n_afterhours_logon	Logons before 07:00 or from 19:00
device	n_device_connect	Removable-device connections
file	n_file_events	File operations
	n_file_to_removable	File operations whose target is removable media
http	n_http_events	Web requests
	n_http_upload	Web uploads
	n_distinct_domains	Distinct web hosts visited
email	n_email_sent	Emails sent ("Send" rows only)
	n_email_external	Sent emails with at least one external recipient
	n_email_attachments	Attachments on sent emails
	total_email_size	Total bytes of sent email

Ground truth. The official answer archive ships the answer keys of every CERT release, so the ingest filters to r6.2, which has exactly five insiders. Each malicious log line is mapped to its user and calendar day, which gives exact day-level labels: 44 malicious user-days. Labelling a user's entire window from first to last malicious event would instead mark benign days between sessions as malicious, and penalise every detector for not flagging them.

Chronological split. The data is split by date at 70%: 990 765 training user-days up to 28 December 2010 and 403 245 test user-days from 29 December 2010. A random split would let a detector learn from days that follow the ones it is scored on. The split has a consequence that shapes everything that follows. Classifying insiders by the span of their malicious activity (longer than three weeks is low-and-slow) gives three burst insiders and two low-and-slow ones. All three burst insiders act before the split and both low-and-slow insiders act after it (Table 1). The test window therefore contains 38 malicious days, all low-and-slow, and the training window contains six, all burst. The supervised baselines must learn the malicious class from six examples of a different kind of attack. Moving the split to put a burst insider in the test window would leave the supervised baselines with no positive training

rows at all, so the constraint is reported rather than engineered away. It also means the test window measures exactly the behaviour SQ1 asks about.

5.2 The Per-User Behavioural Baseline Framework

The framework has the three components the proposal specifies. Algorithm 1 gives the procedure.

Profile Builder. For every user, the per-feature mean, standard deviation and the 25th, 50th, 75th, 90th and 99th percentiles are computed from that user's training-window days. The percentiles are stored so that an explanation can say what a typical and a busy day look like for this person.

Role-Aware Normaliser. The same statistics are computed for every role group and for the whole population. A user with fewer than 14 days of history (the cold-start threshold) is measured against their role group, and a role absent from the training window falls back to the population. The reference frame is exposed as a single *scope* parameter (`population`, `role` or `personal`), so the three frames share one code path and differ in nothing else. This is what makes the ablation in Section 6.4 a clean test of the reference frame.

Anomaly Scorer. Each user-day becomes a deviation vector, z_f or z_f^{robust} per feature, measured against the row's reference profile. A scale of zero, for a feature the user has never exhibited, is floored at one, so the first-ever removable-media write registers as a deviation of one per file rather than a division by zero. One unsupervised model is fitted on the training deviation vectors and scores the test vectors. Because those vectors already express "how unusual for this person", a single global model behaves as a per-user detector, and no per-user model has to be trained or stored. LOF and One-Class SVM are quadratic in the training set, so they are fitted on a seeded subsample of 50 000 training rows. Profiles are always built on the full training window, and every test row is scored.

Algorithm 1 Per-user framework: fit and score.
Input: training user-days D_{train}, test user-days D_{test}, scope, statistic, anomaly model M, cold-start threshold $h_{\min}$
Output: one anomaly score per test user-day, and its explanation

```
# Profile Builder + Role-Aware Normaliser
P_user = stats_by(D_train, "user")  # mean, std,
P_role = stats_by(D_train, "role")  # p25..p99
P_pop  = stats(D_train)

def reference(row):           # scope = personal
    if history(row.user) >= h_min:
        return P_user[row.user]
    if row.role in P_role:
        return P_role[row.role]
    return P_pop

def deviations(rows):
    c, s = centre_scale(reference, rows)
    s[s <= 0] = 1.0           # never-seen feature
    return (rows.X - c) / s

# Anomaly Scorer: one model on deviations
M.fit(subsample(deviations(D_train), 50000))
Z = deviations(D_test)
score   = -M.decision_function(Z)
explain = top_k(abs(Z), k=3)
```

The ranking always uses the model's native score. An early version min-max scaled scores into $[0, 1]$ against training percentiles and clipped them. That collapsed every test row above the training range to exactly 1.0, which is the entire top of the ranking and the only part that matters at a 0.1% budget, so the framework returned 174 indistinguishable rows when asked for four. The bounded score shown in the dashboard is now a display-only calibration that stays strictly ordered with the native score.

5.3 Comparison Set

Every detector implements one interface: it is fitted on the training split and returns one score per test row, higher meaning more suspicious. No detector sets its own threshold. Table 3 lists the families.

The population unsupervised detectors are what make the headline comparison mean something. `pop-ocsvm` and the framework's `peruser-ocsvm-personal-zscore` run the same One-Class SVM with the same parameters

Table 3: The detector families in the comparison.

Family	Detectors	Purpose
Floors	`random`, `volume`	A detector that cannot beat ranking users by total activity has not earned its complexity
Population, supervised	`rf`, `dt`, `gb`, `lr`	The four classifier families Noever compares [4]; `rf` is the proposal's baseline. SMOTE-balanced, raw features
Population, unsupervised	`pop-iforest`, `pop-lof`, `pop-ocsvm`	The framework's own algorithms on population-standardised raw features: the control that isolates the reference frame
Per-user framework	`peruser-<alg>-<scope>-<stat>`	The framework across 3 algorithms, 3 scopes, 2 statistics, with and without a magnitude channel

on the same rows. They differ only in whether each row is expressed relative to the population or relative to the user. Without that control, a framework that beat the Random Forest would have changed two things at once, the algorithm and the reference frame.

5.4 Evaluation Protocol

Operating point. Every detector is evaluated at an alert budget of 0.1% of test user-days, which is 403 alerts over five months across 4 000 users, about three a day. Budgets from 0.01% to 5% are swept to show the trade-off.

Ties. Ties are the normal case on this data. A Random Forest trained on six positives assigns probability zero to most rows. The first implementation flagged every row tied at the cut, which let the Random Forest spend 40 360 alerts against a budget of 404. Ties are now broken by a seeded random permutation, so exactly k rows are flagged, and the choice among equals does not depend on the detector or on the row order. The share of a detector's budget decided this way is reported as its *tie fraction*.

Metrics. The proposal's metrics are reported at the budget: recall, precision, F1, the false positive rate overall and in the high-privilege subgroup, and ROC-AUC. Average precision is the primary ranking metric. Four more were forced by the benchmark:

- **Insider-level recall**, whether each insider was surfaced at all. Day-level recall over 38 positives moves in steps of 2.6 points; whether the person was found is what an analyst acts on.
- **High-privilege alert share**, the share of the alert queue spent on high-privilege users. These users are 2.4% of the test rows, so a small subgroup false positive rate can still mean they dominate the queue. The alert share is what alert fatigue is made of.
- **Alerts ahead**, a threshold-free measure: the number of user-days ranked above an insider's best-ranked malicious day. This is how far down the queue an analyst must read to reach that insider, and it is the investigation-effort proxy for SQ5.
- **Detection delay**, the days between an insider's first malicious day and the first malicious day flagged at the budget. The problem statement is about how long insiders go undetected, and this measures it directly.

High-privilege users are identified from the LDAP role. In r6.2 this subgroup is exactly the ITAdmin role, 100 users. Every metric is computed on the full test split, never a sample: a 5 000-row sample of 403 245 rows contains 0.47 malicious days in expectation.

5.5 Follow-up Studies

Seed robustness. The headline comparison uses one seed. The seed study re-runs seven detectors under ten seeds. These are the framework, its role-scope and Isolation Forest variants, the two population anomaly controls, the Random Forest and the volume floor. Each seed re-draws the fit subsample, the forests, SMOTE and the tie-break, while the split and features stay fixed. The framework is compared with each other detector on average precision, recall and high-privilege alert share by paired two-sided Wilcoxon signed-rank tests, Holm-Bonferroni corrected across all 18 comparisons at $\alpha = 0.05$.

History length (SQ3). To find how much history a personal baseline needs, each user's training data is cut to their most recent N active days, for $N \in \{3, 7, 14, 30, 60, 90, 120, 180\}$ and the full window (248 active days

on average). This simulates an organisation that has been collecting logs for N days when monitoring starts. The framework is refitted on the truncated data and scored on the unchanged test window. The cold-start threshold is lowered to N where necessary, so every user is genuinely measured against an N-day personal profile rather than silently falling back to their role. Two properties are measured. *Stability* has two parts: how far the N-day per-user means sit from the full-history means, in full-history standard deviations, and the share of the test window's *benign* user-days that show any feature beyond $|z| = 3$ under the N-day profile. The latter is the false-deviation load an analyst would inherit. *Usefulness* is detection at the 0.1% budget, compared with the role baseline, which needs no personal history at all. Both the One-Class SVM and Isolation Forest scorers are swept under three seeds.

Explanation fidelity (SQ5). Every answer-key row is a verbatim copy of a malicious log line. Replaying those rows through the aggregator's own per-event handler gives, for each malicious user-day, exactly which features the malicious events changed. This is ground truth for "what the insider did that day", with no hand-made mapping. Each of the 38 malicious test days is explained five ways, each the top three features an analyst would be shown, and every explanation is scored against this ground truth:

- the framework's (largest $|z|$ against the user's own history);
- the same against the role group;
- the same against the population, the control that isolates the reference frame from the explanation method;
- per-row SHAP values of the proposal's Random Forest, ranked by absolute contribution;
- the same SHAP values restricted to features that push towards an alert, since a negative value is a reason against one.

The metrics are hit@1 (the top-named feature is one the insider used), hit@3 and precision@3. They are compared with the analytic rate of a random three-feature explanation, $1 - \binom{13-g}{3} / \binom{13}{3}$ for g relevant features.

5.6 Implementation and Parameters

The system is written in Python with pandas, NumPy, scikit-learn [28], imbalanced-learn (SMOTE), SHAP and SciPy, and has a Streamlit dashboard. Table 4 lists the defaults. All experiments ran on a consumer laptop

(14 cores, 32 GB RAM, no GPU) under Linux and Python 3.14. Every run writes a manifest recording the dataset, split, features, seed, platform and elapsed time, so each table in this report traces back to the run that produced it.

Table 4: Default configuration.

Parameter	**Description**	**Default**
Scope	Reference frame for deviations	`personal`
Statistic	Deviation form	z-score
Anomaly Scorer	Unsupervised model on deviations	One-Class SVM (RBF, $\gamma =$ `scale`)
Contamination / ν	Expected anomalous share	0.02
Cold-start threshold	Days before a personal profile is trusted	14
Fit subsample	Training rows for LOF / One-Class SVM	50 000 (seeded)
Isolation Forest	Trees	200
LOF	Neighbours	25
Random Forest	Trees, depth	300, unlimited
SMOTE	Neighbours	$\min(5, n_{\text{pos}} - 1)$
Alert budget	Share of test rows flagged	0.1% (403 alerts)
Low-and-slow threshold	Span of malicious activity	> 21 days
Seed	All stochastic components	42 (ten in the seed study)

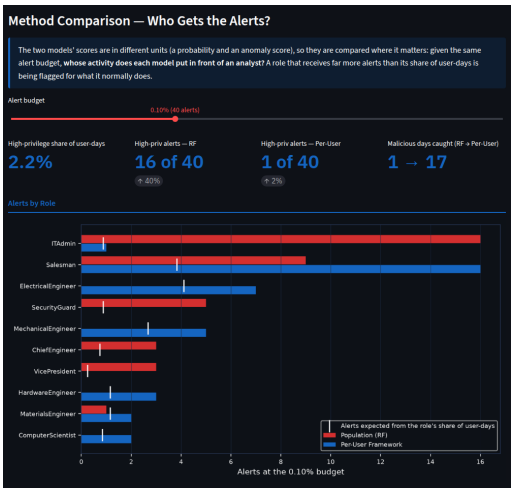
Parameter selection. The anomaly algorithm and the feature set were chosen empirically, as the proposal requires, from the 45-detector ablation in Section 6.4. That ablation was run on a reproducible 400-user, role-stratified cohort that contains all five insiders. The other parameters are library defaults or follow from the data. The contamination rate of 0.02 is close to the alert budgets of interest. The 14-day cold-start threshold is two working weeks, and SQ3 tests it directly.

Prototype dashboard. The proposal promised a prototype that a non-technical security manager could use. The Streamlit dashboard has six pages.

Risk Explorer plots a user's daily risk under both models together with each model's alert line, counts the alerts each model raises on that user, and explains every flagged day against the user's own typical and 90th-percentile day (Figure 2a). *Method Comparison* shows who each model spends its alert budget on, role by role, against each role's share of user-days (Figure 2b). *Evaluation* scores the whole test window at a chosen budget, and *Findings* presents the full-population results reported here. *Overview* and *About* complete the set. To stay interactive, the dashboard runs on the 400-user cohort, which contains all five insiders.



(a) Risk Explorer for CMP2946: the framework (blue) sits on its alert line through the scenario, and each alert names what changed against the user's own normal.



(b) Method Comparison: at 40 alerts the Random Forest spends 16 on IT administrators, whose fair share is about one; the framework spends one.

Figure 2: The prototype dashboard (400-user cohort, 0.1% budget).

5.7 Validation of the Implementation

The repository carries 58 automated regression tests that need no CERT data. Most of them pin a specific way a comparison at a one-in-31 000 positive rate can report a number that looks like a result and is not one. Examples are a bounded score that clips and ties the top of the ranking, an alert budget that tied scores overspend a hundredfold, and metrics computed on a sample that contains no positives. Others cover a rolling feature that reads across

the split (verified by a check that is itself tested to fail on a forward-looking window), a magnitude channel that duplicates the population detector, and an answer-key replay that shifts columns. A further set of slow tests rederives features directly from the 93 GB of raw logs and checks them against the aggregated table. Writing the follow-up studies uncovered one more defect: SMOTE was always seeded with 42 regardless of the run's seed, contrary to the documented design. It was fixed before the seed study ran.

6 Results and Discussion

Unless stated otherwise, results are for the full 4 000-user population and the headline seed (42), at the 0.1% alert budget: 403 alerts over 403 245 test user-days, which contain 38 malicious days belonging to two insiders. Section 6.5 shows that the conclusions hold across ten seeds.

6.1 Headline Comparison

Table 5 compares the framework with every population detector and both floors.

The framework flags 21 of the 38 malicious test days (recall 0.553), against 8 for the Random Forest (0.211). It is the only detector to surface *both* test-window insiders. Its alerts are 553 times more likely to be malicious than a random alert. It spends 3.5% of its alert queue on IT administrators against the Random Forest's 22.6%, and it trains in 7.9 seconds against 447.

The same-algorithm control isolates the cause. `pop-ocsvm` runs the identical One-Class SVM on the identical rows, with each row expressed relative to the population instead of relative to the user. It finds 3 malicious days and spends 20.8% of its alerts on administrators. Changing nothing but the reference frame therefore raises recall from 0.079 to 0.553 and cuts the high-privilege alert share sixfold.

Figure 3 shows that the ordering is not an artefact of the chosen budget. From 0.05% upwards the framework's recall exceeds every population detector's, and at 0.5% (about 13 alerts a day) it reaches 0.84. The Random Forest stays at 0.21 from 0.05% to 1% because its ranking runs out: beyond its few confident rows, every remaining test row has probability zero, and 85% of its 0.5% budget is a random draw among ties. At the smallest budget, 40 alerts, the framework finds nothing. Section 6.9 traces this to a saturation property of the RBF One-Class SVM.

Table 5: Full population, 0.1% alert budget (403 alerts), seed 42. "Days" is the number of the 38 malicious test days flagged; "HP share" is the share of alerts spent on high-privilege (ITAdmin) users, who make up 2.43% of test user-days. Best value per column in bold. Lift (precision over the base rate) is 553 for the framework and 211 for the Random Forest.

Detector	**AP**	**ROC-AUC**	**Days**	**Insiders**	**HP share**	**Fit (s)**
Per-user framework						
`peruser-ocsvm-personal-zscore`	0.0383	**0.967**	**21**	**2/2**	3.5%	7.9
`peruser-ocsvm-role-zscore`	**0.0433**	0.675	3	1/2	0.7%	7.1
`peruser-iforest-personal-zscore`	0.0030	0.921	3	1/2	7.7%	7.7
Population, unsupervised (same algorithms, population reference)						
`pop-ocsvm`	0.0364	0.446	3	1/2	20.8%	2.7
`pop-iforest`	0.0009	0.935	0	0/2	49.4%	3.7
`pop-lof`	0.0003	0.831	0	0/2	**0.0%**	17.7
Population, supervised (SMOTE-balanced)						
`rf` (proposal baseline)	0.0077	0.605	8	1/2	22.6%	446.8
`lr`	0.0284	0.864	2	1/2	9.7%	6.9
`gb`	0.0003	0.379	1	1/2	44.4%	15.0
`dt`	0.0001	0.500	0	0/2	2.7%	70.5
Floors						
`volume`	0.0284	0.821	2	1/2	4.0%	**0.0**
`random`	0.0001	0.445	0	0/2	2.0%	0.0

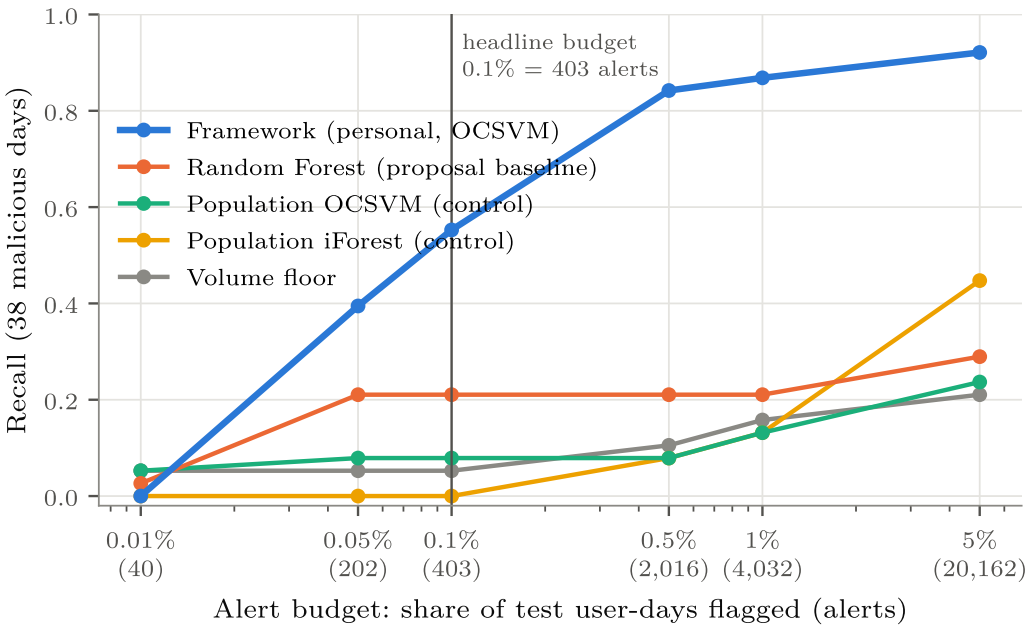


Figure 3: Recall of the 38 malicious test days at alert budgets from 40 to 20 162 alerts.

6.2 SQ1: Low-and-Slow Detection

Every malicious day in the test window belongs to a low-and-slow insider, so the headline recall answers SQ1 directly: the per-user baseline finds 2.6 times as many low-and-slow malicious days as the population Random Forest (21 versus 8) and 7 times as many as the same algorithm with a population reference (21 versus 3).

Figure 4 shows why. CMP2946's scenario (job-hunting followed by thumb-drive exfiltration) never produces a day that is extreme for the organisation. They browse about 160 pages a day before and during the scenario, which is ordinary across 4 000 users. Against their own history, however, the scenario's days are abrupt departures. The framework places 18 of their 29 malicious days inside the alert line, and the population-referenced model places none. CDE1846's malicious days, by contrast, are large bursts of emailed attachments that are extreme for anyone. Both reference frames catch the same 3 of their 9 days, and the per-user baseline neither gains nor loses on them. The framework's advantage is therefore concentrated exactly where the research question locates it: behaviour that is unusual for the person but not for the population.

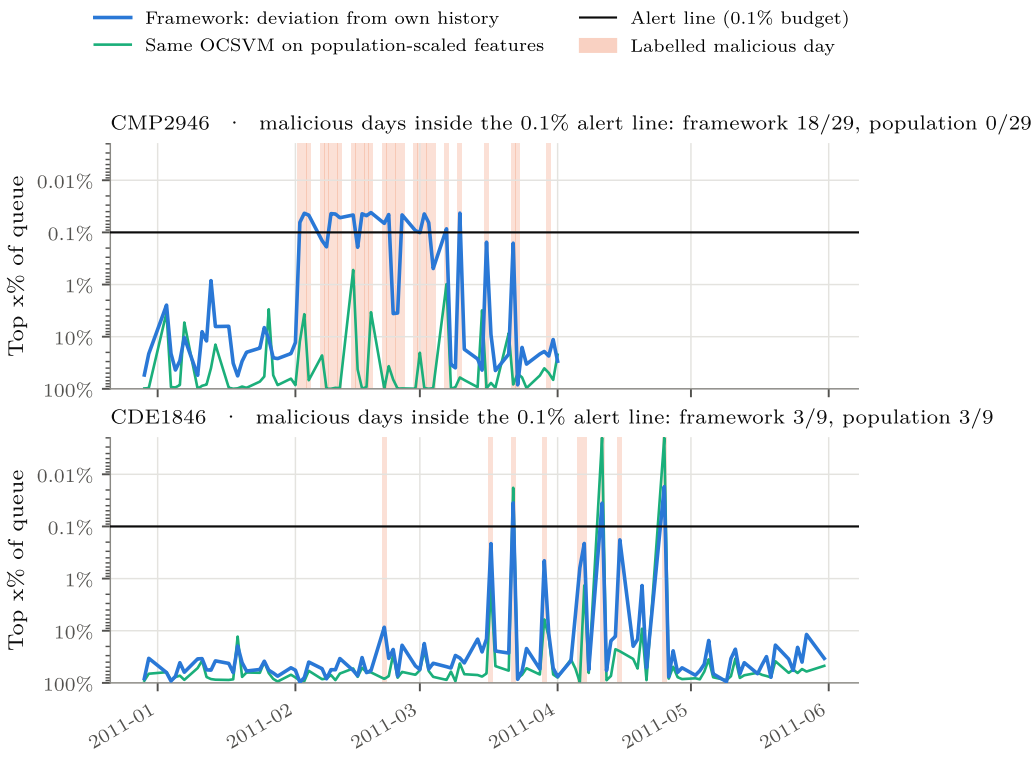


Figure 4: Daily queue position of the two low-and-slow insiders over the test window, under the framework and under the same One-Class SVM with a population reference. Shaded days are labelled malicious; the black line is the 0.1% alert budget.

Queue depth and detection delay. Table 6 reports, for each insider and across all ten seeds of Section 6.5, how many user-days rank above the insider's first appearance in each detector's queue, and how long after their first malicious day they were first flagged. Every population detector buries one of the two insiders thousands of alerts deep. The Random Forest ranks CMP2946's scenario highly, but places CDE1846 about 24 000 alerts down, roughly sixty times its budget, and never flags them. The same-algorithm control and the volume floor do the reverse. The framework is the only detector that brings both insiders inside its first 200 alerts in every seed. It flagged CMP2946 on the first day of their scenario. It flagged CDE1846 29 days in, on the first of their malicious days to involve a burst of emailed attachments, and so did every detector that flagged them at all.

Table 6: How far down each detector's queue an analyst must read to reach each insider (user-days ranked above the insider's best malicious day; median over ten seeds, with range), and days from the insider's first malicious day to the first flagged one at the 0.1% budget ("never": not flagged in any seed).

	Alerts ahead		Detection delay (days)	
Detector	**CMP2946**	**CDE1846**	**CMP2946**	**CDE1846**
Framework (personal, OCSVM)	186 (168 to 191)	85 (6 to 140)	0	29
Framework, role scope	5 626	0	never	29
Framework, iForest scorer	1 239	80 (40 to 218)	never	29
Population OCSVM (control)	2 175	0 (0 to 1)	never	29
Population iForest	3 595	1 414	never	never
Random Forest (baseline)	45 (19 to 78)	24 367 (9 093 to 77 971)	0	never
Volume floor	30 040	0	never	49

6.3 SQ2: False Positives on High-Privilege Users

High-privilege users are 2.43% of the test user-days. Figure 5 shows how much of each detector's queue they occupy. Population detectors over-spend on them by an order of magnitude: 49.4% of `pop-iforest`'s alerts, 44.4% of `gb`'s, 22.6% of the Random Forest's and 20.8% of `pop-ocsvm`'s go to administrators doing their jobs. Every per-user variant without the magnitude channel sits between 0.5% and 16%, and the role-scoped variants are lowest (0.7% and 1.7%). In subgroup false positive rate terms, the Random Forest raises a false alert on 0.93% of administrator-days and the framework on 0.14%, a 6.5-fold reduction.

The effect is again isolated by holding the algorithm fixed. Isolation Forest spends 49.4% of its alerts on administrators with a population reference and 7.7% with a personal one. One-Class SVM goes from 20.8% to 3.5%. Expressing activity relative to the individual, not the algorithm, removes the bias.

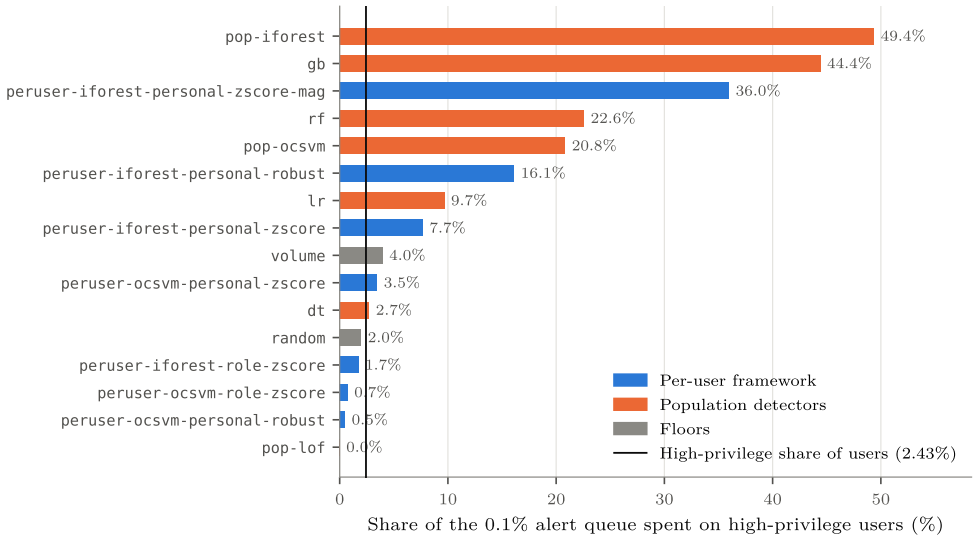


Figure 5: Share of each detector's 403 alerts spent on high-privilege users. The vertical line is their share of the test population.

The problem is not confined to the role the proposal anticipated. Executives are also legitimately unusual: the population One-Class SVM raises a false alert on 18.7% of the President's test days and 4.0% of Vice-Presidents', and Isolation Forest on 15.9% and 2.9%. The framework's rates are 0.9% and 0%. A detector that only special-cases a list of administrator roles would miss this; one that measures each user against themselves does not need the list.

Two results temper the claim. First, the role-scoped framework suppresses administrator alerts best (0.7%) but finds only 3 malicious days, so the personal scope is the better trade-off and not the lowest false-positive option. Second, feeding the anomaly model population-scaled raw magnitudes alongside the deviations, intended to stop trivially small but personally unusual days from crowding the queue, raises Isolation Forest's administrator share from 7.7% to 36.0%. It reintroduces precisely the population bias the framework exists to remove, and is kept in the ablation as a negative result. The framework does not use it.

6.4 Ablation: Reference Frame, Statistic, Algorithm and Features

The proposal requires the anomaly algorithm to be chosen empirically. Table 7 summarises the 45-detector ablation, which crosses scope, statistic, algorithm and the magnitude channel on a reproducible, role-stratified 400-user cohort that keeps all five insiders (40 360 test user-days, 40 alerts at 0.1%).

Table 7: Ablation on the 400-user cohort: average precision / recall at 0.1% (40 alerts), z-score statistic. The population-scope cells reproduce `pop-ocsvm`, `pop-iforest` and `pop-lof` exactly, confirming that the three scopes share one code path.

Anomaly Scorer	**Population**	**Role**	**Personal**
One-Class SVM	0.046 / 0.08	0.086 / 0.08	**0.292 / 0.50**
Isolation Forest	0.009 / 0.00	0.022 / 0.08	0.059 / 0.08
LOF	0.002 / 0.00	0.002 / 0.00	0.027 / 0.00
Variants of the personal scope			
Robust statistic (OCSVM / iForest)	0.030 / 0.03	and	0.044 / 0.08
+ magnitude channel (OCSVM / iForest)	0.173 / 0.37	and	0.047 / 0.08

Three conclusions follow. The reference frame matters for every algorithm: the personal scope gives the highest average precision in all three rows, by a factor of six to fourteen over the population scope. The algorithm matters more than expected: One-Class SVM on personal z-scores is best by a factor of five, LOF finds nothing in any configuration while being the slowest model, and Isolation Forest sits in between. The robust statistic, which was expected to make low-and-slow drift more visible by resisting spikes, did not help the best configuration. A likely reason is that counts which are zero on most days have an interquartile range of zero, and flooring that scale at one discards the user's real variability.

Temporal features. Adding causal 7- and 30-day rolling means for every feature (39 features in all) helps one scorer and hurts another (Table 8). Isolation Forest, which samples one feature per split and tolerates redundancy, gains ninefold in average precision. One-Class SVM loses most of its recall, as expected of an RBF kernel when thirteen features become 39 highly cor-

related ones and distances dilute. The population detectors gain nothing, which is itself informative: behavioural drift is informative only relative to an individual's own baseline. The algorithm and the feature set therefore have to be selected together. The best configuration overall is One-Class SVM on personal z-scores with the base features.

Table 8: Effect of causal rolling-window features on the full population.

	Average precision		Recall at 0.1%	
Detector	**base**	**+ rolling**	**base**	**+ rolling**
`peruser-iforest-personal-robust`	0.0041	**0.0378**	0.079	0.079
`peruser-ocsvm-personal-zscore`	**0.0383**	0.0125	**0.553**	0.053
`volume`	0.0284	0.0281	0.053	0.053
`pop-iforest`	0.0009	0.0006	0.000	0.000

6.5 Robustness Across Seeds

Every comparison so far uses one seed. Table 9 and Figure 6 repeat the seven key detectors under ten seeds, with every random component re-drawn each time: the fit subsample, the forests, SMOTE and the tie-break.

Table 9: Mean ± standard deviation over ten seeds, full population, 0.1% budget. "Days" is the range of malicious days flagged across the seeds.

Detector	AP	Recall	Days	Insiders	HP share	Fit (s)
`peruser-ocsvm-personal-zscore`	0.0363 ± 0.0011	**0.547 ± 0.011**	**20 to 21**	**2/2**	3.4 ± 0.2%	10.4
`peruser-ocsvm-role-zscore`	**0.0473 ± 0.0049**	0.079 ± 0.000	3	1/2	**0.5 ± 0.1%**	9.5
`peruser-iforest-personal-zscore`	0.0029 ± 0.0008	0.076 ± 0.008	2 to 3	1/2	7.4 ± 1.5%	8.0
`pop-ocsvm`	0.0360 ± 0.0063	0.079 ± 0.000	3	1/2	20.4 ± 1.7%	2.9
`pop-iforest`	0.0009 ± 0.0001	0.000 ± 0.000	0	0/2	47.5 ± 7.3%	3.6
`rf`	0.0049 ± 0.0027	0.163 ± 0.046	3 to 8	1/2	21.3 ± 2.5%	486.4
`volume`	0.0284 ± 0.0000	0.053 ± 0.000	2	1/2	4.0 ± 0.0%	0.0

The framework's result is stable. It found 20 or 21 of the 38 malicious days in every seed, and both insiders in every seed. The Random Forest ranged from 3 to 8 days, and its single-seed headline (8 days) was its best draw. The paired tests confirm the picture. Seventeen of the eighteen framework-versus-detector comparisons went the same way in all ten seeds, fifteen of them in the framework's favour. Each gives an exact two-sided Wilcoxon

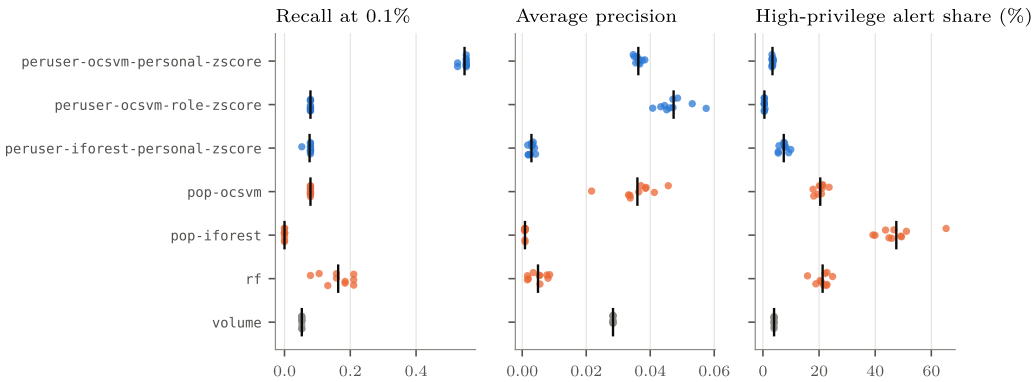


Figure 6: Each detector under ten seeds. Dots are seeds; the black tick is the mean.

$p = 0.002$, and $p_{\text{Holm}} = 0.035$ after correcting for all eighteen, the strongest result a ten-seed design with eighteen tests can produce. The framework has significantly higher recall than every other detector. It has a significantly lower high-privilege alert share than every population detector, the volume floor and its own Isolation Forest variant. It also has significantly higher average precision than the Random Forest, both Isolation Forest detectors and the volume floor.

Three comparisons do not favour the framework, and they bound what is claimed. Against its same-algorithm control, average precision is a tie: five seeds each way, $p = 1$. Both rank one of CDE1846's attachment-burst days at or near the very top of the queue, which dominates average precision, so the per-user frame's advantage lies in recall and in where the second insider lands, not in the top-ranked alert. Against the role-scoped variant, the framework loses in all ten seeds on both average precision and administrator alert share, while finding seven times as many malicious days. Role normalisation is the stronger suppressor of false alerts; personal baselines are the stronger detector.

6.6 SQ3: How Much History a Personal Baseline Needs

Figure 7 and Table 10 show the history-length sweep. The two views of stability agree. With three days of history, a third of all benign test days show some feature beyond $|z| = 3$, because a few days cannot estimate a user's spread. That share falls to 13% at 14 days and 8.7% at 30 days,

and then flattens, reaching 5.9% with the full history. The per-user means converge in the same way: by 30 days, 99.5% of user-feature means lie within half a standard deviation of their full-history value.

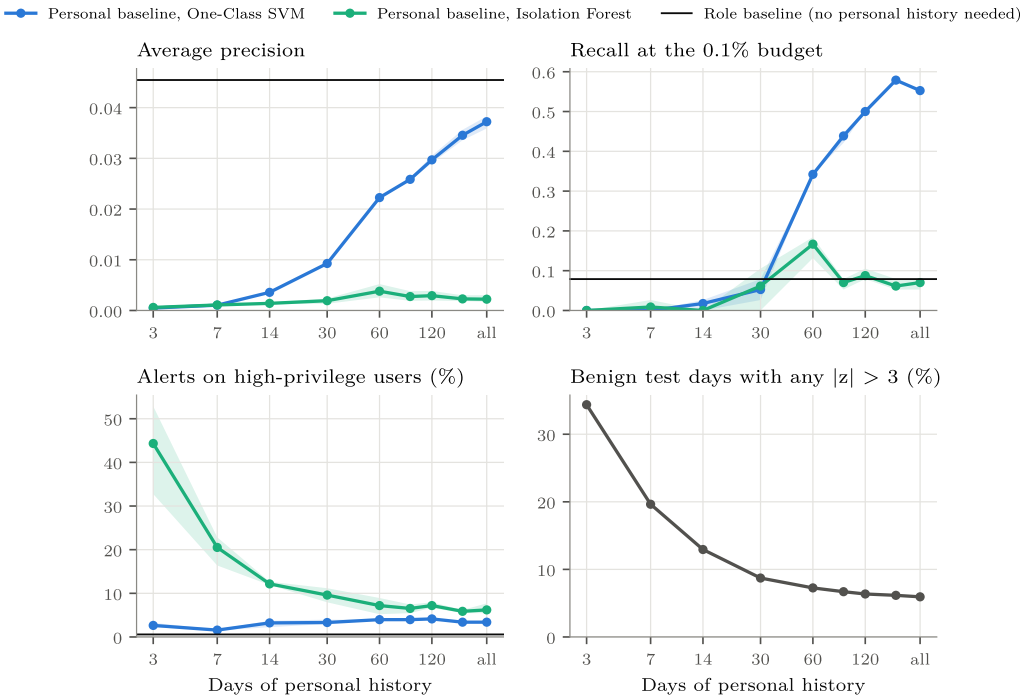


Figure 7: Detection and stability against days of personal history (most recent active days before monitoring starts). Lines are means over three seeds; bands span the seeds.

Detection needs more history than stability does. Below 30 days the personal baseline finds almost nothing and is worse than simply using the role baseline (recall 0.079). At 60 active days, about three months of working days, recall jumps to 0.34 and both insiders are surfaced. Gains continue to roughly 120 to 180 days and then plateau. Below 13 days the anomaly model also has fewer than its usual 50 000 training rows, so the shortest histories conflate profile quality with training-set size. From 14 days upward the model's training size is constant, and the trend is attributable to the profiles alone.

The answer to SQ3 is therefore about 30 active days for a baseline that is *stable*, and about 60 for one that is *useful*. Two practical consequences follow. The proposal's 14-day cold-start threshold is too low: users should remain on their role baseline until about 60 days of history are available.

Table 10: History-length sweep, One-Class SVM personal baseline (means over three seeds). "False-deviation days" is the share of benign test days with any feature beyond $|z| = 3$.

History	AP	Recall	Insider recall	HP share	False-deviation days
3 days	0.0005	0.000	0.00	2.7%	34.4%
7 days	0.0010	0.000	0.00	1.6%	19.6%
14 days	0.0036	0.018	0.33	3.2%	12.9%
30 days	0.0093	0.053	0.50	3.3%	8.7%
60 days	0.0223	0.342	1.00	4.0%	7.3%
90 days	0.0259	0.439	1.00	4.0%	6.7%
120 days	0.0297	0.500	1.00	4.1%	6.4%
180 days	0.0345	0.579	1.00	3.4%	6.2%
All (248)	0.0372	0.553	1.00	3.4%	5.9%
Role baseline	0.0454	0.079	0.50	0.6%	

And insufficient history is not merely ineffective, it is harmful for the wrong scorer. With three days of history the Isolation Forest variant spends 44% of its alerts on administrators (Figure 7, bottom left), because an underestimated spread makes naturally variable users look anomalous. A short personal baseline recreates the population failure; the One-Class SVM variant is robust to this (1.6 to 4.1% throughout).

6.7 SQ4: Computational Cost

Table 11 reports measured cost on a laptop with 14 cores, 32 GB of RAM and no GPU.

Building 4 000 personal profiles, 46 role profiles and the population profile from 990 765 rows, and fitting the anomaly model, takes under eight seconds. The profiles are vectorised group-by aggregations, and only one anomaly model is fitted regardless of the number of users, so cost grows linearly with rows and not with users. The Random Forest is 57 times slower to train, because SMOTE adds 990 753 synthetic positives to the six real ones, doubling the training set to 1 981 518 rows. The framework is slower to *score*, at 0.025 ms per user-day, but a 4 000-person organisation produces about 4 000 user-days a day, which score in a tenth of a second. The expensive step is the one-off reduction of the raw logs, which streams 93 GB in 3.3

Table 11: Computational cost on the full population (990 765 training and 403 245 test user-days).

Detector	**Fit (s)**	**ms/row**	**Peak (GB)**
Framework (personal, One-Class SVM)	7.9	0.025	1.3
Framework, Isolation Forest scorer	7.7	0.008	1.3
Population One-Class SVM	2.7	0.024	0.2
Random Forest (proposal baseline)	446.8	0.002	1.9
Logistic Regression	6.9	0.0003	0.5
One-off: raw logs to user-day table (93 GB)	198		bounded
One-off: user-day table to frozen dataset	3		

minutes with bounded memory. A role-stratified 400-user subset trains end to end in 20 seconds. Per-user baselining is therefore feasible on consumer hardware, and cheaper to train than the population baseline it replaces.

6.8 SQ5: Context for the Analyst

A user study of investigation time was outside the project's reach. SQ5 is therefore answered with two measurable proxies. The first is queue depth: how far down the ranked queue an analyst must read before reaching each insider (Section 6.2). The second, reported here, is whether the reason each system gives for a malicious day names what the insider actually did.

Figure 8 scores the top three features of each explanation against the answer key's ground truth for all 38 malicious test days. SHAP is scored in two readings: ranked by absolute contribution, and restricted to features that push towards an alert. The framework's explanation names a feature the malicious events actually changed as its top reason on 94.7% of days, against a random-explanation rate of 29.6%. With the same method but a population reference the rate is 52.6%, and against the role group 39.5%. The explanation method is identical in all three; only the reference frame differs. On the 21 malicious days the framework actually flagged, its top-named reason is correct every time.

How the Random Forest's SHAP explanations perform depends on how

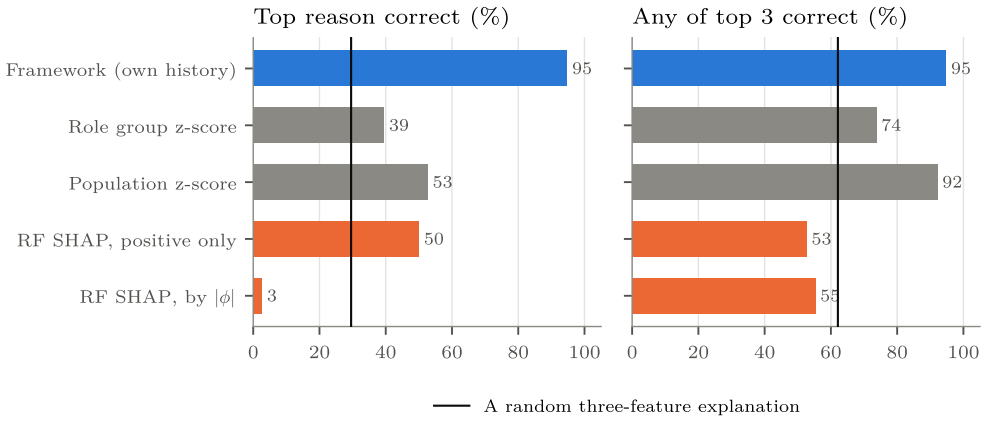


Figure 8: How often each explanation names a feature the insider's malicious events actually changed that day, over the 38 malicious test days. The horizontal line is the analytic rate of a random three-feature explanation.

they are read. Ranked by absolute contribution, they name a relevant feature first on only 2.6% of malicious days, far below the 29.6% of a random explanation, and on none of the 8 days the forest itself flagged. The forest assigns most low-and-slow days a probability near zero, so its largest SHAP values are reasons *against* an alert. Reading only the features that push towards an alert raises the rate to 50.0% over all malicious days and to 87.5% on the forest's own 8 alerts. When the forest does raise an alert, it can usually say why, provided its explanation is read for positive contributions. The dashboard therefore shows only the positive contributions.

Two differences from the framework remain. The first is coverage. The framework raised 21 correct alerts and its top-named reason was right for all 21, while the forest raised 8. Across all 38 malicious days, even the positive reading of SHAP includes a relevant feature in its top three less often than a random explanation would (52.6% against 62.1%). The forest learned the malicious class from the burst insiders in the training window, and frames every day in their terms: web uploads, device connections and removable-media writes. The second is context. A SHAP value is a contribution to a probability and says nothing about what is normal for the person. A deviation says what the value is today, what it usually is for this user, and how far apart the two are.

The insider-level breakdown mirrors the detection result. On CDE1846's days, which are extreme for anyone, every z-score explanation is correct. On CMP2946's, the framework's top reason is correct 93% of the time, against 38% with a population reference and 21% with a role reference. Personal

context matters for exactly the behaviour that is unusual only for the person.

Table 12 shows what this means at the analyst's desk. On CMP2946's highest-scored malicious day, the framework reports that web requests rose to 173, from a user who made exactly 162 on every training day. With a spread of zero, floored at one, that is a deviation of 11. To the population this is a 1.8-standard-deviation day, unremarkable among 4 000 users. On CDE1846's, it reports eight file operations against a usual zero (12 standard deviations), which the population reference rates at 1.4. That is the trace of the insider logging on to other users' machines to search for files, alongside the emailed attachments that every reference frame sees. The forest flagged neither day, and its SHAP values for both are small and negative: they explain why it did not alert.

Table 12: What each system tells the analyst about the two insiders' highest-scored malicious days. "Usual" and "p90" are the user's own median and 90th-percentile day. Rows are the features the framework names first, plus the email features for CDE1846.

Feature	**Today**	**Usual**	**p90**	z **(own)**	z **(population)**	**RF SHAP**
CMP2946, 18 February 2011 (job hunting, before the thumb-drive phase)						
`n_http_events`	173	162	162	**+11.0**	+1.8	−0.04
`n_email_sent`	12	6	8	+3.2	+4.4	−0.05
`n_distinct_domains`	27	23	26	+1.2	+0.3	−0.00
CDE1846, 25 April 2011 (another user's PC, files emailed to a personal address)						
`total_email_size`	48.6 MB	1.4 MB	5.1 MB	**+19.4**	+21.5	−0.03
`n_email_attachments`	48	1	6	+16.7	+20.0	−0.03
`n_email_external`	18	1	3	+15.2	+11.7	−0.07
`n_file_events`	8	0	1	**+12.2**	+1.4	−0.07

The answer to SQ5 is that a per-user deviation tells the analyst what changed, by how much relative to this person's own normal, and what their normal is. Measured against ground truth, that reason was right on 95% of malicious days and on every alert the framework raised. The Random Forest's SHAP explanations were right for most of its own, far fewer, alerts when read for positive contributions, but no better than chance across the malicious days as a whole, and they carry no notion of the user's normal. Whether this shortens investigations in practice needs the user study proposed as future work. The evidence here is that the per-user framework both puts insiders nearer the top of the queue and hands the analyst the right place to start.

6.9 Unexpected and Negative Results

A volume floor beats most population baselines. Ranking user-days by total activity, with no model at all, matches Logistic Regression's average precision (0.0284) and beats the Random Forest's (0.0077). It also beats Gradient Boosting, the Decision Tree, Isolation Forest and LOF. Scenario 2's thumb-drive days are simply busy days. Results on this benchmark that do not report such a floor should be read with caution.

Two supervised baselines fail outright. Trained to purity on SMOTE-balanced data, the Decision Tree emits exactly two distinct scores. It therefore has no ranking, and 99.5% of its budget is a coin toss. Gradient Boosting reaches a ROC-AUC of 0.379, worse than random. Both follow from learning a malicious class from six burst examples and being tested on low-and-slow behaviour.

ROC-AUC and average precision mislead in opposite directions. `pop-iforest` has a ROC-AUC of 0.935, the third highest in Table 5, yet finds no malicious day and spends half its queue on administrators. It orders the benign mass well and the top badly. Conversely, the role-scoped framework has the highest average precision (0.0433) but finds 3 days to the personal framework's 21. Average precision is dominated by the very top of the ranking, where one correct alert ranked first outweighs twenty ranked a little lower. No single number suffices; recall and insider-level recall at the budget, the budget sweep, and queue depth are all reported for that reason.

The RBF One-Class SVM saturates at the top. Far from every support vector, the RBF kernel sum is zero and the One-Class SVM's decision function equals its offset ρ. In the test window, 147 user-days receive exactly this maximum score, 3 of them malicious. At the 0.1% budget all 147 are included and the effect is invisible, but at 40 alerts the selection among them is a seeded coin toss (tie fraction 1.0) that happened to draw no malicious day. Ordering the saturated group by the magnitude of the deviation vector would fix this, and is left as future work.

6.10 Threats to Validity

Conclusion validity. The test window contains 38 malicious days, but they belong to only two people. The seed study controls for algorithmic randomness; it cannot control for which insiders the benchmark happens to

contain. Insider-level claims rest on $n = 2$, and day-level recall moves in steps of 2.6 points. The results are evidence about these two low-and-slow scenarios, not an estimate of performance on insiders in general.

Internal validity. The chronological split puts every burst insider in the training window, so the supervised baselines learn from six examples of a different kind of attack. This handicaps them, and is why the population *unsupervised* controls, which see no labels either, are the fairer comparison. The framework beats those too. More seriously, the anomaly algorithm was selected on the 400-user cohort, whose test window contains the same two insiders as the full test window. The choice of One-Class SVM over the proposal's Isolation Forest default is therefore informed by test-window behaviour, and the headline recall is optimistic. The conclusions that do not depend on that choice are stated separately for this reason. With the a-priori Isolation Forest scorer, the personal reference frame still cuts the administrator alert share from 49.4% to 7.7%, but finds no more malicious days than the same-algorithm population control does.

Construct validity. Labels are day-level, so a flagged day counts as a hit even if the analyst would need to find the malicious events within it. The alert budget is applied over the whole test window, which is retrospective: a deployed system must choose each day's cut-off without seeing the future. The high-privilege subgroup is defined by role and in r6.2 is exactly one role. The only high-privilege insider (PLJ1771, scenario 3) acts in the training window, so this evaluation cannot show whether per-user scoring would also suppress a *malicious* administrator.

External validity. CERT r6.2 is synthetic, generated by behavioural models of one fictitious organisation [11]. Its users are far more regular than real employees: CMP2946 makes roughly 160 web requests on most days. Regularity flatters personal baselines, because small departures stand out. Results on real, noisier logs, and on other CERT releases, may be weaker.

7 Conclusion

This research asked whether a lightweight per-user behavioural baseline could detect insider activity that a population-level detector misses, while spending fewer alerts on legitimately busy users, on hardware a resource-constrained organisation already owns. The framework profiles each user from their own

history, falls back to their role group when history is short, and scores each user-day's deviation vector with one unsupervised model. It was evaluated on the full CERT r6.2 dataset against the population Random Forest that prior work identifies as strongest, against the same anomaly algorithms with a population reference, and against trivial floors.

Answers to the sub-questions.

- **SQ1 (low-and-slow detection).** At a 0.1% alert budget the per-user baseline found 21 of 38 low-and-slow malicious days and both insiders, against 8 days and one insider for the Random Forest and 3 days for the same algorithm with a population reference. The gain is concentrated where the hypothesis predicts: on the insider whose behaviour was unusual for themselves but ordinary for the organisation. Across ten seeds it found 20 or 21 days every time, against 3 to 8 for the Random Forest, and it was the only detector to bring both insiders within the first 200 alerts of its queue.

- **SQ2 (high-privilege false positives).** Population detectors spent 21 to 49% of their alerts on IT administrators, who are 2.4% of user-days. The framework spent 3.5% and the role-scoped variant 0.7%. Holding the algorithm fixed shows the reference frame, not the model, removes the bias. The same holds for executives, whom no role list would have anticipated.

- **SQ3 (history needed).** A personal baseline is stable after about 30 active days and useful after about 60. Below that it is worse than the role baseline, and for the Isolation Forest scorer it recreates the administrator false-positive problem. The cold-start threshold should therefore be about 60 days, not the 14 assumed in the proposal.

- **SQ4 (cost).** The framework trains on 990 765 user-days in under eight seconds on a laptop, 57 times faster than the SMOTE-balanced Random Forest, because one model is fitted regardless of the number of users. The one-off reduction of 93 GB of logs takes 3.3 minutes in bounded memory.

- **SQ5 (analyst context).** A per-user deviation tells the analyst what changed, by how much relative to this person's normal, and what that normal is. Against answer-key ground truth, its top-named reason was correct on 94.7% of malicious test days. The same explanation with a population reference managed 52.6%. The Random Forest's SHAP

values managed 50.0% when read for positive contributions, and 2.6% when ranked by absolute value, below the 29.6% of a random guess. The framework also places both insiders within the first 190 alerts of its queue, where the Random Forest leaves one of them about 24 000 alerts deep. Whether this shortens real investigations needs a user study.

Contributions. The work contributes a working, reproducible framework and data pipeline. It also contributes a controlled evaluation design (alert budgets, same-algorithm controls, a volume floor, tie-aware metrics, queue depth and detection delay) that other work on this benchmark can reuse. Three follow-up studies extend the proposal's metrics: seed robustness with corrected significance tests, a history-length study, and an answer-key-grounded measure of explanation fidelity. Finally, the report documents the negative results, which are as informative as the positive ones.

Limitations. The evidence comes from one synthetic dataset and two test-window insiders. The anomaly algorithm was selected on data containing those insiders. The only malicious administrator falls outside the test window, and the alert budget is set retrospectively. Section 6.10 discusses each.

Future work. Five directions follow directly from the findings:

- **A daily operating point.** Choose each day's alerts from that day's scores, so the evaluation matches what a deployed system can actually do.
- **Ordering within the saturated region.** Break the One-Class SVM's ties at its maximum score by the magnitude of the deviation vector.
- **Adaptive baselines.** Update profiles as new days arrive, and switch from role to personal reference at the roughly 60 days SQ3 identifies.
- **More data.** Select the algorithm on a held-out release and evaluate on others (r4.2, r5.2), or on real, consented logs.
- **A user study.** Measure whether the per-user explanations reduce analysts' investigation time, which this report could only approximate.

Final remarks. Judging each employee against their own normal, rather than the organisation's, found a slow insider the population model could not see. It stopped busy administrators and executives from dominating the alert queue, and it ran in seconds on a laptop. The approach is not a replacement for a security operations centre. It is evidence that the core idea of enterprise behaviour analytics is within reach of organisations that cannot buy one.

References

[1] IBM Security, "Cost of a data breach report 2024," tech. rep., IBM Corporation, 2024.

[2] D. M. Cappelli, A. P. Moore, and R. F. Trzeciak, *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes*. Addison-Wesley Professional, 2012.

[3] I. Homoliak, F. Toffalini, J. Guarnizo, Y. Elovici, and M. Ochoa, "Insight into insiders and IT: A survey of insider threat taxonomies, analysis, modeling, and countermeasures," *ACM Computing Surveys*, vol. 52, no. 2, pp. 30:1–30:40, 2019.

[4] D. Noever, "Classifier suites for insider threat detection." arXiv preprint arXiv:1901.10948, 2019.

[5] D. C. Le and A. N. Zincir-Heywood, "Evaluating insider threat detection workflow using supervised and unsupervised learning," in *IEEE Security and Privacy Workshops (SPW)*, pp. 270–275, 2018.

[6] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *31st USENIX Security Symposium*, 2022.

[7] Microsoft, "Advanced threat detection with user and entity behavior analytics (UEBA) in Microsoft Sentinel." Microsoft Learn. `https://learn.microsoft.com/en-us/azure/sentinel/identify-threats-with-entity-behavior-analytics`. Accessed October 2026, 2026.

[8] A. Tuor, S. Kaplan, B. Hutchinson, N. Nichols, and S. Robinson, "Deep learning for unsupervised insider threat detection in structured cybersecurity data streams," in *AAAI Workshop on Artificial Intelligence for Cyber Security*, pp. 224–231, 2017.

[9] D. C. Le, N. Zincir-Heywood, and M. I. Heywood, "Analyzing data granularity levels for insider threat detection using machine learning," *IEEE Transactions on Network and Service Management*, vol. 17, no. 1, pp. 30–44, 2020.

[10] B. Lindauer, "Insider threat test dataset." Carnegie Mellon University, KiltHub. `https://doi.org/10.1184/R1/12841247.v1`, 2020.

[11] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *IEEE Security and Privacy Workshops (SPW)*, pp. 98–104, 2013.

[12] S. Yuan and X. Wu, "Deep learning for insider threat detection: Review, challenges and opportunities," *Computers & Security*, vol. 104, p. 102221, 2021.

[13] P. J. Rousseeuw and M. Hubert, "Robust statistics for outlier detection," *WIREs Data Mining and Knowledge Discovery*, vol. 1, no. 1, pp. 73–79, 2011.

[14] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[15] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pp. 413–422, 2008.

[16] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 93–104, 2000.

[17] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[18] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[19] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 233–240, 2006.

[20] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[21] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017.

[22] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[23] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[24] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[25] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[26] F. Yuan, Y. Cao, Y. Shang, Y. Liu, J. Tan, and B. Fang, "Insider threat detection with deep neural network," in *Computational Science – ICCS 2018*, vol. 10860 of *Lecture Notes in Computer Science*, Springer, 2018.

[27] G. Shoderu, S. Baror, A. Modupe, and S. Makura, "Digital forensic readiness to mitigate insider threats in the SaaS cloud environment," in *Proceedings of the 21st International Conference on Cyber Warfare and Security (ICCWS)*, vol. 21, 2026. doi:10.34190/iccws.21.1.4508.

[28] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

A Reproducing the Results

Every number in this report is produced by the commands below, run from the repository root. Each writes its tables and a `run.json` manifest under `results/`.

```
python script/dataset.py          # verify + extract r6.2 and answers
python script/aggregate.py        # 93 GB of logs -> user-day table
python script/prepare.py          # + roles, labels, time split
python script/prepare.py --cohort 400

python script/experiment.py --preset core          # headline
python script/experiment.py --preset core --temporal
python script/experiment.py --preset baselines
python script/experiment.py --preset full \
    --dataset data/artifacts/dataset_cohort400_seed42.parquet

python script/studies.py history    # SQ3
python script/studies.py explain    # SQ5
python script/studies.py seeds      # ten seeds, Wilcoxon + Holm

python report/make_figures.py       # every data figure
pytest -m "not slow"                # 58 regression tests
streamlit run app.py                # the dashboard
```